Understanding diffusion model from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

October 7, 2025

Feynman's words

What I cannot create, I do not understand.

Outline

Generate samples from given distribution

Given a distribution $p(x)$, how to sample from the distribution?

- Uniform distribution & Gaussian distribution: easy sampling
- Mixture Gaussian: determine which Gaussian to sample & sample from that Gaussian
- A general sample function?
  - Langevin Markov Chain Monte Carlo sampling (Langevin MCMC)
  - Stein Variational Gradient Descent (SVGD)
  - ...

Score function is all you need?

A typical process of generating new samples from modeling the data distribution

1. To find the probability function $p(x)$, model $p(x;\theta)$.
2. Normalization distribution $p(x;\theta) = \frac{1}{C(\theta)} e^{-E(x;\theta)}$ with $C(\theta) \equiv \int_x e^{-E(x;\theta)} dx$.
3. Generating samples from above sampling methods.

Recall the Langevin Markov Chain Monte Carlo sampling

$$x_t = x_{t-1} + \frac{\Delta t}{2} \nabla_x \log p(x_{t-1};\theta) + \sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0,1). \tag{1}$$

Here $\nabla_x \log p(x_{t-1};\theta) = -\nabla_x E(x;\theta)$ is free of the normalization $C(\theta)$.
Define the score function

$$S(x;\theta) \equiv \nabla \log p(x;\theta). \tag{2}$$

More about Langevin MCMC and the score function

Continuous form of the Langevin MCMC

- $\Delta t \to 0$

$$dx = \frac{1}{2}\nabla_x \log p(x)dt + dB_t, \tag{3}$$

  where dB_t is a Brownian motion.
- Given $p(x) = \frac{1}{C}e^{-E}$, we have

$$dx = -\frac{1}{2}\nabla_x E dt + dB_t \tag{4}$$

Langevin MCMC is overdamped Langevin dynamics

- Langevin dynamics (MD)

$$\ddot{x} = -\nabla_x E - \gamma\dot{x} + dB_t, \tag{5}$$

- Given $\gamma\dot{x} \gg \ddot{x}$, we have the overdamped Langevin equation

$$dx = -\frac{1}{\gamma}\nabla_x E + \frac{1}{\gamma}dB_t. \tag{6}$$

Outline

Matching the score by neural networks

Recall the definition

$$S \equiv \nabla_x \log p(x), \tag{7}$$

The key is to matching the score by a neural network

$$J_{ESM} = \frac{1}{2}\mathbb{E}_p \left[\| s(x;\theta) - \frac{\partial \log p(x)}{\partial x} \|^2 \right], \tag{8}$$

where J_{ESM} is called Explicit Score Matching loss. However, it is not tractable to compute $\frac{\partial \log p(x)}{\partial x}$ from data.

Where should we go?

Implicit Score Matching (ISM) loss

Implicit Score Matching loss (Aapo, 2005) was developed to make a tractable score matching

$$J_{ISM} \equiv \mathbb{E}_p \left[\frac{1}{2} \|s(x;\theta)\|^2 + \nabla \cdot s(x;\theta) \right] = J_{ESM} - C. \tag{9}$$

ISM Proof: Expansion of ESM

Proof.

First, we expand the ESM loss:

$$J_{ESM} = \frac{1}{2}\mathbb{E}_p\left[\|s(x;\theta) - \frac{\partial \log p(x)}{\partial x}\|^2\right] \tag{10}$$

$$= \frac{1}{2}\mathbb{E}_p\left[\|s\|^2\right] - \mathbb{E}_p[s \cdot \frac{\partial \log p}{\partial x}] + \frac{1}{2}\mathbb{E}_p\left[\|\frac{\partial \log p}{\partial x}\|^2\right] \tag{11}$$

The key insight is to simplify the cross term using integration by parts. □

Proof (Cont).

The cross term can be simplified:

$$-\mathbb{E}_p[s \cdot \frac{\partial \log p}{\partial x}] = -\int_x ps \cdot \nabla \log p dx \tag{12}$$

$$= -\int_x ps \cdot \frac{\nabla p}{p} dx = -\int_x s \cdot \nabla p dx \tag{13}$$

$$= -\int \nabla \cdot (ps) dx + \int p \nabla \cdot s dx = \mathbb{E}_p[\nabla \cdot s] \tag{14}$$

ISM Proof: Final Result

Proof (Cont).

Therefore, we have:

$$J_{ISM} = J_{ESM} - C \tag{15}$$

where $C \equiv \frac{1}{2}\mathbb{E}_p \left[\| \frac{\partial \log p}{\partial x} \|^2 \right]$ is a constant independent of θ.

This makes ISM tractable since we don't need to compute the true score function.

Denoising Score Matching (DSM) loss

However, the ISM score is not very stable to optimize, with two reasons

1. Expectation is performed on the whole distribution;
2. The loss is negative decreasing to $-C$ with a great quantity, e.g. $-1e5$.

Denoising Score Matching (DSM) loss (Vincent, 2011) is developed to solve this problem

$$J_{DSM} = \frac{1}{2}\mathbb{E}_{p(x,\tilde{x})}\left[\|s(\tilde{x};\theta) - \nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2\right]. \tag{16}$$

And we can prove that $J_{DSM} = J_{ESM} + C$.

DSM Proof: Expansion

Proof.

We expand the DSM loss:

$$J_{DSM} \equiv \frac{1}{2}\mathbb{E}_{p(x,\tilde{x})}\left[\|s(\tilde{x};\theta) - \nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2\right] \tag{17}$$

$$= \frac{1}{2}\mathbb{E}_{p(\tilde{x})}\left[\|s(\tilde{x})\|^2\right] - \mathbb{E}_{p(x,\tilde{x})}[s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x}|x)] \tag{18}$$

$$+\frac{1}{2}\mathbb{E}_{p(x,\tilde{x})}[\|\nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2] \tag{19}$$

The key step is to show that the cross terms of J_{ESM} and J_{DSM} are equal. □

DSM Proof: Cross Term Equivalence

Proof (Cont).

We need to show:

$$\mathbb{E}_{p}(\tilde{x})[s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x})] = \mathbb{E}_{p(\tilde{x},x)}\left[s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x}|x)\right] \tag{20}$$

Starting from the left side:

$$= \int_{\tilde{x}} p(\tilde{x}) s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x}) d\tilde{x} \tag{21}$$

$$= \int_{\tilde{x}} s(\tilde{x}) \cdot \nabla_{\tilde{x}} p(\tilde{x}) d\tilde{x} \tag{22}$$

DSM Proof: Cross Term Equivalence (Cont)

Proof (Cont).

Continuing the derivation:

$$= \int_{\tilde{x}} \int_{x} p(x) s(\tilde{x}) \cdot \nabla_{\tilde{x}} p(\tilde{x}|x) d\tilde{x} dx \tag{23}$$

$$= \int_{\tilde{x}} \int_{x} p(\tilde{x}|x) p(x) s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x}|x) d\tilde{x} dx \tag{24}$$

$$= \mathbb{E}_{p(\tilde{x},x)} \left[s(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p(\tilde{x}|x) \right] \tag{25}$$

This completes the proof of cross term equivalence.

DSM Proof: Final Result

Proof (Cont).

Therefore, we have:

$$J_{DSM} = J_{ESM} - \frac{1}{2}\mathbb{E}_p\left[\|\nabla_{\tilde{x}} \log p(\tilde{x})\|^2\right] + \frac{1}{2}\mathbb{E}_{p(x,\tilde{x})}[\|\nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2] \tag{26}$$

Since the last two terms are constants independent of θ, we have $J_{DSM} = J_{ESM} + C$ where C is a constant.

DSM is all you need?

Problem arises when $\tilde{x} \to x$. We can prove that when Gaussian noised added by $p(\tilde{x}|x) = N(\alpha x, \sigma^2)$,

$$J_{DSM} \to \infty, \text{ if } \tilde{x} \to x. \tag{27}$$

Proof.

Given the Gaussian noise added, we have

$$\nabla_{\tilde{x}} \log p(\tilde{x}|x) = \nabla_{\tilde{x}} \log e^{-\frac{(\tilde{x}-\alpha x)^2}{2\sigma^2}} = -\frac{\tilde{x} - \alpha x}{\sigma^2}. \tag{28}$$

And we can show that the following formula goes to infinity

$$\begin{aligned} &\mathbb{E}_{p(x,\tilde{x})}[\|\nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2] = \mathbb{E}_{p(x,\tilde{x})}[\|\frac{\tilde{x} - \alpha x}{\sigma^2}\|^2] \\ &= \mathbb{E}_{p(x)}\mathbb{E}_{p(\tilde{x}|x)}[\|\frac{\tilde{x} - \alpha x}{\sigma^2}\|^2] = \mathbb{E}_{\epsilon \sim N(0,1)}[\|\frac{\epsilon}{\sigma}\|^2] \to \infty \text{ if } \sigma \to 0. \end{aligned} \tag{29}$$

Finally, we know that

$$J_{DSM} = J_{ESM} + C \to \infty \text{ as } C \to \infty \text{ when } \tilde{x} \to x. \tag{30}$$

Outline

Revisit the adding noise from x to $\tilde{x}$

Gaussian transition kernel for adding noise from x to $\tilde{x}$

$$\tilde{x} = \alpha x + \sigma\epsilon, \tag{31}$$

which equivalent to the Gaussian transition kernel

$$p(\tilde{x}|x) = N(\tilde{x}; \alpha x, \sigma^2), \tag{32}$$

where $\epsilon \sim N(0, 1)$ and

$$N(\tilde{x}; \alpha x, \sigma^2) = Ce^{-\frac{\|\tilde{x}-\alpha x\|^2}{2\sigma^2}}. \tag{33}$$

And

$$\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}. \tag{34}$$

Questions arise: how can we get x given $\tilde{x}$?

Tweedie's Formula

Tweedie's Formula (Bayes statistics) states (Robbins, 1956)

Theorem

Given random variables x, y, $y \sim N(x, \sigma^2 I)$, i.e., $p(y|x) = N(x, \sigma^2)$, the expectation of x could be given by

$$\mathbb{E}[x|y] = y + \sigma^2 \nabla \log p(y). \tag{35}$$

Let

$$x \leftarrow \alpha x, \quad y \leftarrow \tilde{x}, \tag{36}$$

we have

$$s \equiv \nabla_{\tilde{x}} \log p(\tilde{x}) = -\frac{\tilde{x} - \alpha \mathbb{E}[x|\tilde{x}]}{\sigma^2} = -\frac{1}{\sigma}\mathbb{E}[\frac{\tilde{x} - \alpha x}{\sigma}|\tilde{x}] = -\frac{\mathbb{E}[\epsilon|\tilde{x}]}{\sigma}, \tag{37}$$

where we have used the fact $\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}$.

Another insight: from the definition of the score S

Recall that adding noise by $\tilde{x} = \alpha x + \sigma\epsilon$,

$$p(\tilde{x}|x) = Ce^{-\frac{\|\tilde{x}-\alpha x\|^2}{2\sigma^2}}. \tag{38}$$

Hence

$$s(\tilde{x}) = \nabla_{\tilde{x}} \log p(\tilde{x}) = \frac{\nabla_{\tilde{x}} p(\tilde{x})}{p(\tilde{x})} = \frac{\int_x \nabla_{\tilde{x}} p(\tilde{x}|x)p(x)dx}{p(\tilde{x})}, \tag{39}$$

From Eq. (38), we have

$$s(\tilde{x}) = \frac{\int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x)p(\tilde{x}|x)p(x)dx}{p(\tilde{x})} = \int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x)p(x|\tilde{x})dx, \tag{40}$$

leading to

$$s(\tilde{x}) = \mathbb{E}_x[\nabla_{\tilde{x}} \log p(\tilde{x}|x)|\tilde{x}] = \mathbb{E}_x[-\frac{\tilde{x}-\alpha x}{\sigma^2}|\tilde{x}] = -\frac{\tilde{x}-\alpha\mathbb{E}[x|\tilde{x}]}{\sigma^2}. \tag{41}$$

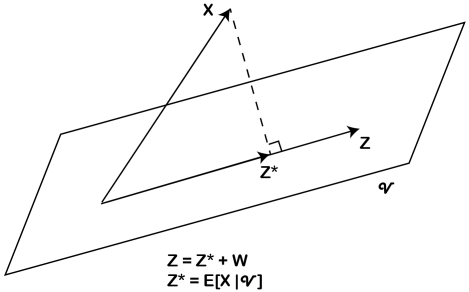
Theoretical Support: Conditional Expectation

Theorem

Let X be an integrable random variable. Then for each σ-algebra $\mathcal{V}$ and $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X|\mathcal{V})$ solves the least square problem

$$argmin_{Z \in \mathcal{V}} \|Z - X\|,$$

where $\|Z\| = \left(\int Z^2 dP\right)^{\frac{1}{2}}$.



Setting

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space and $L^2 := L^2(\Omega, \mathcal{F}, \mathbb{P})$ with $\langle U, V \rangle = \mathbb{E}[UV]$. Fix a sub-$\sigma$-field $\mathcal{G} \subset \mathcal{F}$ and denote the closed subspace

$$\mathcal{V} := L^2(\mathcal{G}) = \{Z \in L^2 : Z \text{ is } \mathcal{G}\text{-measurable}\}.$$

Claim. For $X \in L^2$,

$$Z^* := \mathbb{E}[X \mid \mathcal{G}] \in \mathcal{V} \quad \text{and} \quad Z^* = \arg\min_{Z \in \mathcal{V}} \mathbb{E}[(Z - X)^2].$$

Proof of the claim

For any $Z \in \mathcal{V}$,

$$\langle X - Z^*, Z \rangle = \mathbb{E}[(X - Z^*)Z] = \mathbb{E}(\mathbb{E}[(X - Z^*)Z \mid \mathcal{G}]) = \mathbb{E}(Z\,\mathbb{E}[X - Z^* \mid \mathcal{G}]) = 0.$$

Hence $X - Z^* \perp \mathcal{V}$.

Let any $Z \in \mathcal{V}$ be written as $Z = Z^* + W$ with $W \in \mathcal{V}$. Then

$$\|X - Z\|_2^2 = \|X - (Z^* + W)\|_2^2 = \|X - Z^*\|_2^2 + \|W\|_2^2 \geq \|X - Z^*\|_2^2,$$

because $\langle X - Z^*, W \rangle = 0$. Equality $\Leftrightarrow W = 0 \Leftrightarrow Z = Z^*$, proving uniqueness and the minimizer claim.

Revisit several models

1. Score model
$$Loss_{score} = \|s_\theta(\tilde{x}) - \frac{\epsilon}{\sigma}\|^2; \tag{42}$$
2. X prediction model (denoising model, $x0$ model)
$$Loss_{x_0} = \|x_\theta(\tilde{x}) - x\|^2, \tag{43}$$
with
$$s(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} \tag{44}$$
where $x_\theta(\tilde{x}) \to x_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[x|\tilde{x}]$;
3. ϵ prediction model
$$Loss_\epsilon = \|\epsilon_\theta(\tilde{x}) - \epsilon\|^2, \tag{45}$$
with
$$s(\tilde{x}) = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}, \tag{46}$$
where $\epsilon_\theta(\tilde{x}) \to \epsilon_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[\epsilon|\tilde{x}]$.

The consistency of the three modeling

As stated before, we have the following connection in the optimal (θ^*) case

$$s_{\theta^*}(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}, \tag{47}$$

by the above conditional expectation.
Hence, in the real modeling of designing the loss, we use the connection of three models without reaching the optimal parameter

$$s_{\theta}(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta}(\tilde{x})}{\sigma^2} = -\frac{\epsilon_{\theta}(\tilde{x})}{\sigma}, \tag{48}$$

and substitute each other form to the loss definition above, we can recover all of the losses given by different models.

Outline

Key Identity: Jacobian Switch (chain rule between $\mathbf{x}_0$ and $\mathbf{x}_t$)

The Gaussian transition density is

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \alpha_t\,\mathbf{x}_0,\ \sigma_t^2\mathbf{I}\big) \propto \exp\Big(-\tfrac{\|\mathbf{x}_t-\alpha_t\,\mathbf{x}_0\|^2}{2\sigma_t^2}\Big).$$

A basic but crucial identity:

$$\nabla_{\mathbf{x}_t} q(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{1}{\alpha_t}\,\nabla_{\mathbf{x}_0} q(\mathbf{x}_t \mid \mathbf{x}_0), \qquad (\alpha_t > 0).$$

Proof sketch. Differentiating the exponent w.r.t. $\mathbf{x}_t$ and w.r.t. $\mathbf{x}_0$ shows $\nabla_{\mathbf{x}_t}\|\mathbf{x}_t - \alpha_t\mathbf{x}_0\|^2 = 2(\mathbf{x}_t - \alpha_t\mathbf{x}_0)$, $\nabla_{\mathbf{x}_0}\|\mathbf{x}_t - \alpha_t\mathbf{x}_0\|^2 = -2\alpha_t(\mathbf{x}_t - \alpha_t\mathbf{x}_0)$, thus the factor $-1/\alpha_t$.

Theorem 1: Score as Conditional Force Expectation

Statement. With $p(\mathbf{x}_0) \propto e^{-E(\mathbf{x}_0)/k_B T}$ and $\mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}$,

$$\boxed{\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0|\mathbf{x}_t}\left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T}\right],} \qquad \mathbf{F}(\mathbf{x}_0) = -\nabla_{\mathbf{x}_0} E(\mathbf{x}_0).$$

Derivation.

$$\begin{aligned}
\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) &= \frac{\nabla_{\mathbf{x}_t} \int p(\mathbf{x}_0)\, q(\mathbf{x}_t \mid \mathbf{x}_0)\, \mathrm{d}\mathbf{x}_0}{p(\mathbf{x}_t)} \\
&= \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0)\, \nabla_{\mathbf{x}_t} q(\mathbf{x}_t \mid \mathbf{x}_0)\, \mathrm{d}\mathbf{x}_0
\end{aligned}$$

Theorem 1: Derivation (Cont)

Proof (Cont).

Using the Jacobian switch identity $(*)$:

$$\begin{aligned}
&\overset{(*)}{=} \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0) \Big(-\frac{1}{\alpha_t} \Big) \nabla_{\mathbf{x}_0} q(\mathbf{x}_t \mid \mathbf{x}_0) \, \mathrm{d}\mathbf{x}_0 \\
&= -\frac{1}{\alpha_t} \frac{1}{p(\mathbf{x}_t)} \int q(\mathbf{x}_t \mid \mathbf{x}_0) \nabla_{\mathbf{x}_0} p(\mathbf{x}_0) \, \mathrm{d}\mathbf{x}_0
\end{aligned}$$

Theorem 1: Derivation (Final)

Proof (Cont).

$$= -\frac{1}{\alpha_t} \int \underbrace{\frac{p(\mathbf{x}_0) q(\mathbf{x}_t \mid \mathbf{x}_0)}{p(\mathbf{x}_t)}}_{p(\mathbf{x}_0 \mid \mathbf{x}_t)} \nabla_{\mathbf{x}_0} \log p(\mathbf{x}_0) \, \mathrm{d}\mathbf{x}_0$$

$$= \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 \mid \mathbf{x}_t} \left[\frac{-\nabla_{\mathbf{x}_0} E(\mathbf{x}_0)}{k_B T} \right] = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 \mid \mathbf{x}_t} \left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T} \right]$$

Experiments on first 1, 000 frames for LJ data

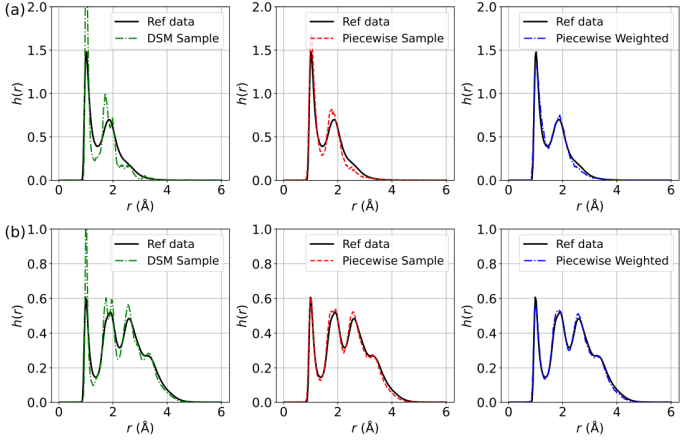


Figure 1: The distribution of interatomic distances $h(r)$ for LJ potential using biased training data with a sample size of 500. (a) Comparison of $h(r)$ for DSM, Piecewise, and Piecewise Weighted losses (from left to right) against reference data for LJ-13; (b) Comparison of $h(r)$ for DSM, Piecewise, and Piecewise Weighted losses for LJ-55.

Experiments on first 5, 000 frames for MD reference trajectories

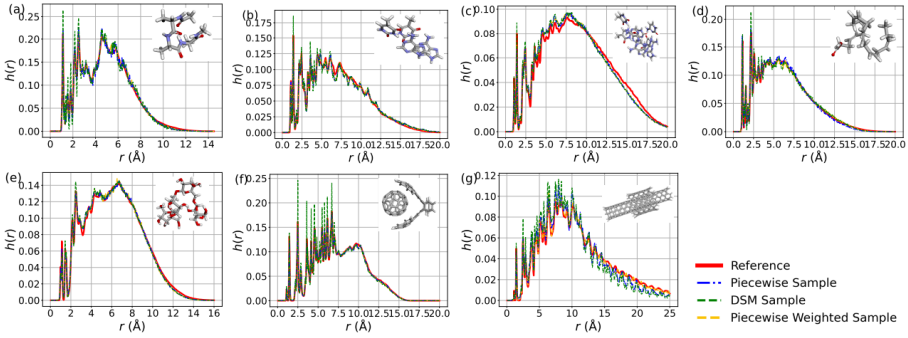


Figure 3: The distribution of interatomic distances (r(Å)) for (a) Ac-Ala3-NHMe, (b) DNA base pair (AT-AT), (c) DNA base pair (AT-AT-CG-CG), (d) Docosahexaenoic acid, (e) Stachyose, (f) Buckyball catcher, (g) Dw nanotube in MD22 dataset. The insets display the ball-and-stick representations of these molecules.

Outline

From Theory to Practice

Now that we have established the theoretical foundations of score matching and diffusion processes, let's explore how these concepts are implemented in practice through several key approaches:

- **SMLD** (Score Matching Langevin Dynamics)
- **DDPM** (Denoising Diffusion Probabilistic Models)
- **Continuous formulations** and their advantages

Each approach offers different insights into the practical implementation of diffusion-based generative models.

SMLD

Score Matching Langevin Dynamic (SMLD) (Song, 2019) adding noise in a following manner

$$p(\tilde{x}_i|x) = N(\tilde{x}_i; x, \sigma_i^2) \equiv Ce^{-\frac{\|\tilde{x}_i - x\|^2}{2\sigma_i^2}}, \tag{49}$$

where a geometric sequence $\sigma_1 > \sigma_2 > \cdots > \sigma_T \approx 0$ is given to add noise with different level. In a random variable view,

$$\tilde{x}_i = x + \sigma_i \epsilon \tag{50}$$

for different σ_i to get different noised random variable $\tilde{x}_i$.
Training object J_{DSM} is given by

$$J_{DSM} = \frac{1}{2}\mathbb{E}_{\sigma_i}\mathbb{E}_{p(x)}\mathbb{E}_{p(\tilde{x}_i|x)}\left[\|s_\theta(\tilde{x}_i, \sigma_i) + \frac{\tilde{x}_i - x}{\sigma_i^2}\|^2\right]. \tag{51}$$

Anneled Langevin MCMC for sampling

$$x_t = x_{t-1} + \frac{\Delta t}{2} S_\theta(x_{t-1}) + \sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0, 1). \tag{52}$$

DDPM

denoising diffusion probabilistic models (DDPM) (Ho, 2020)

1. Derived from the Evidence Lower BOund (ELBO), Not score matching.
2. First provide adding noise and denoise in the forward and reverse process.

The main procedure

1. Adding noise
$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon; \tag{53}$$
2. Transition kernel
$$p(x_t|x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \tag{54}$$
where $\alpha_t = \prod_{i=1}^{t}\sqrt{1-\beta_i}$ and $\sigma_t^2 = 1-\alpha_t^2$.

DDPM

The main procedure (Continued)

1. Adding noise
$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon; \tag{55}$$
2. Transition kernel
$$p(x_t|x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \text{ i.e., } x_t = \alpha_t x_0 + \sigma_t \epsilon, \tag{56}$$
where $\alpha_t = \prod_{i=1}^{t}\sqrt{1-\beta_i}$ and $\sigma_t^2 = 1-\alpha_t^2$.
3. Training Loss
$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{p(x)}\mathbb{E}_{p(x_t|x)}\left[\|\epsilon_\theta(x_t,t)-\epsilon\|^2\right], \tag{57}$$
where we have used the approximation $x_0 = \frac{x_t - \sigma_t\epsilon}{\alpha_t}$.
4. Sampling
$$x_{t-1} = \frac{1}{\sqrt{1-\beta_t}}(x_t + \beta_t s_\theta(x_t,t)) + \sqrt{\beta_t}\epsilon_t, \quad t = T, T-1, \cdots, 1. \tag{58}$$

Outline

The continuous version of SMLD

Recall that

$$x_t = x_0 + \sigma_t \epsilon, \tag{59}$$

we can prove that

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon. \tag{60}$$

Proof.

We can prove the formula by a Recurrence

$$\begin{aligned} x_t &= x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon \\ &= x_{t-2} + \sqrt{\sigma_{t-1}^2 - \sigma_{t-2}^2}\epsilon + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon \\ &= x_{t-2} + \sqrt{\sigma_t^2 - \sigma_{t-2}^2}\epsilon \\ &= \ldots = x_0 + \sigma_t \epsilon, \qquad \text{where } \sigma_0 \approx 0. \end{aligned} \tag{61}$$

The continuous version of SMLD

From

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon, \tag{62}$$

we have

$$\begin{aligned} \Delta x_t &= \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}}\sqrt{\Delta t}\epsilon \\ &= \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}}\Delta B_t. \end{aligned} \tag{63}$$

Further, let $\Delta t \to 0$, we have

$$dx = \sqrt{\frac{d\sigma_t^2}{dt}}dB_t. \tag{64}$$

The continuous version of DDPM

Recall that

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon. \tag{65}$$

Similarly

$$x_t \approx (1 - \frac{1}{2}\beta_t)x_{t-1} + \sqrt{\beta_t}\epsilon, \tag{66}$$

hence, we have

$$\begin{aligned} \Delta x_t &= -\frac{1}{2}\tilde{\beta}_t x_{t-1}\Delta t + \sqrt{\tilde{\beta}_t}\sqrt{\Delta t}\epsilon, \quad \text{where } \tilde{\beta}_t = \frac{\beta_i}{\Delta t} \\ &= -\frac{1}{2}\tilde{\beta}_t x_{t-1}\Delta t + \sqrt{\tilde{\beta}_t}\Delta B_t. \end{aligned} \tag{67}$$

Further, let $\Delta t \to 0$, we finally obtain

$$dx = -\frac{1}{2}\tilde{\beta}_t x dt + \sqrt{\tilde{\beta}_t}dB_t. \tag{68}$$

VE & VP

Summarizing the form of continuous DDPM and SMLD, we summarize the adding noise process as

$$dx = f(x,t)dt + g(t)dB_t, \tag{69}$$

where the corresponding distribution $p(x,t)$ satisfying the following Fokker-Planck equation

$$\frac{\partial p(x,t)}{\partial t} + \nabla \cdot (f(x,t)p(x,t)) - \frac{1}{2}g^2(t)\triangle p(x,t) = 0. \tag{70}$$

Training DSM object

$$J_{DSM} = \mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{p(x_0)}\mathbb{E}_{p(x_t|x_0)}\left[\|s_\theta(x_t,t) - \nabla_{x_t}\log p(x_t|x_0)\|^2\right]. \tag{71}$$

VE & VP adding noise

To training the Denoising Score Match loss J_{DSM} above, the key is to add noise by the transition kernel

$$p(\tilde{x}|x) \equiv p(x(t)|x(0)) = N(x(t); \tilde{\alpha}_t x(0), \tilde{\sigma}_t^2), \tag{72}$$

from which we can obtain the noised data

$$x(t) = \tilde{\alpha}_t x(0) + \tilde{\sigma}_t \epsilon, \quad \epsilon \sim N(0, 1). \tag{73}$$

The question becomes: Given VE & VP

$$dx = \sqrt{\frac{d[\sigma^2]}{dt}} dB_t, \quad dx = -\frac{1}{2}\tilde{\beta}_t x dt + \sqrt{\tilde{\beta}_t} dB_t. \tag{74}$$

how to derive $p(x(t)|x(0))$, i.e., $p(\tilde{x}|x)$ for adding noise?

VE & VP Transition kernel

The summarizing form of VE & VP is

$$dx = h(t)xdt + g(t)dB_t, \tag{75}$$

where $h(t) = 0$ for VE and $h(t) = -\frac{1}{2}\tilde{\beta}(t)$ for VP.

Lemma

Let $\mu(t) = \mathbb{E}[x(t)]$ and $\Sigma(t) = \mathbb{E}\left[(x - \mu(t))(x - \mu(t))^T\right]$, we have the following formula

$$\frac{d\mu(t)}{dt} = h(t)\mu(t), \quad \frac{d\Sigma(t)}{dt} = 2h(t)\Sigma(t) + g^2(t)I. \tag{76}$$

The proof of the Lemma can be found in a stochastic differential equation textbook such as Applied Stochastic Differential Equations, Särkkä and Solin, 2019.

VE & VP Transition kernel

Lemma

For VE, the transition kernel is given by the following formula

$$P(x(t)|x(0)) = N(x(t); x(0), \sigma^2(t) - \sigma^2(0)). \tag{77}$$

For VE,

$$\tilde{\alpha}_t \equiv 1, \quad \tilde{\sigma}_t^2 \equiv \sigma^2(t) - \sigma^2(0) \approx \sigma^2(t). \tag{78}$$

Lemma

For VP, the transition kernel is given the following formula

$$p(x(t)|x(0)) = N\left(x(t); x(0)e^{-\frac{1}{2}\int_0^t \tilde{\beta}(s)ds}, 1 - e^{-\int_0^t \tilde{\beta}(s)ds}\right). \tag{79}$$

For VP,

$$\tilde{\alpha}_t \equiv e^{-\frac{1}{2}\int_0^t \tilde{\beta}(s)ds}, \quad \tilde{\sigma}_t^2 \equiv 1 - e^{-\int_0^t \tilde{\beta}(s)ds}. \tag{80}$$

Proof of VE transition kernel

Proof.

For VE, by a simple calculation we have

$$\frac{d\mu(t)}{dt} = 0, \quad \frac{d\Sigma(t)}{dt} = \frac{d[\sigma^2(t)]}{dt}, \tag{81}$$

Hence, it is easy to obtain the following form

$$\mu(t) = \mu(0) = x(0), \quad \Sigma(t) = \sigma^2(t) - \sigma^2(0). \tag{82}$$

Here we have used the fact

$$\mu(0) = x(0), \quad \Sigma(0) = 0 \tag{83}$$

as $x(0)$ is taken as a constant, not a random variable. And we obtain

$$P(x(t)|x(0)) = N(x(t); x(0), \sigma^2(t) - \sigma^2(0)). \tag{84}$$

VP Proof: Differential Equations

Proof.

For VP, we have the differential equations:

$$\frac{d\mu(t)}{dt} = -\frac{1}{2}\beta(t)\mu(t) \tag{85}$$

$$\frac{d\Sigma(t)}{dt} = -\beta(t)\Sigma(t) + \beta(t) \tag{86}$$

We solve these using integrating factors. □

Proof (Cont).

For the mean equation, multiply by $e^{\int_0^t \frac{1}{2}\beta(s)ds}$:

$$\frac{d\mu(t)}{dt}e^{\int_0^t \frac{1}{2}\beta(s)ds} + \frac{1}{2}\beta(t)\mu(t)e^{\int_0^t \frac{1}{2}\beta(s)ds} = 0 \tag{87}$$

For the variance equation, multiply by $e^{\int_0^t \beta(s)ds}$:

$$\frac{d\Sigma(t)}{dt}e^{\int_0^t \beta(s)ds} + \beta(t)\Sigma(t)e^{\int_0^t \beta(s)ds} = \beta(t)e^{\int_0^t \beta(s)ds} \tag{88}$$

VP Proof: Solution

Proof (Cont).

This gives us:

$$\frac{d}{dt}(\mu(t)e^{\int_0^t \frac{1}{2}\beta(s)ds}) = 0 \tag{89}$$

$$\frac{d}{dt}(\Sigma(t)e^{\int_0^t \beta(s)ds}) = \frac{d}{dt}e^{\int_0^t \beta(s)ds} \tag{90}$$

Solving with initial conditions $\mu(0) = x(0)$, $\Sigma(0) = 0$:

$$p(x(t)|x(0)) = N\left(x(t); x(0)e^{-\frac{1}{2}\int_0^t \beta(s)ds}, 1 - e^{-\int_0^t \beta(s)ds}\right) \tag{91}$$

Revisit adding noise

The **equivalence** of the adding noise

1. From the stochastic process, denote the variable x_t, we have

$$dx_t = f(x,t)dt + g(t)dB_t, \tag{92}$$

with corresponding discrete form (Euler-Maruyama scheme)

$$x_{t+\Delta t} = x_t + f(x_t,t)\Delta t + g(t)\sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0,1). \tag{93}$$

2. From transition kernel perspective,

$$p(x_t) = \int_{x_0} p(x_0)p(x_t|x_0)dx_0, \tag{94}$$

where

$$p(x_t|x_0) = N(x_t;\mu_t,\Sigma_t) = N(x_t;\tilde{\alpha}_t x_0, \tilde{\sigma}_t^2) = \frac{1}{C}e^{-\frac{\|x_t-\mu_t\|^2}{2\Sigma_t}}. \tag{95}$$

Training recipe

The training for VE follows the following process (VP is in a similar way)

1. Random choose one data $x \sim p_{data}$;
2. Random sample $t \sim U[0,1]$;
3. Random sample a white noise $\epsilon \sim N(0,1)$;
4. Adding noise
$$x_t = \tilde{\alpha}_t x + \tilde{\sigma}_t \epsilon, \tag{96}$$
where the mean value $\tilde{\alpha}_t x = x(0)$ and standard deviation $\tilde{\sigma}_t = \sqrt{\sigma^2(t) - \sigma^2(0)} \approx \sigma(t)$ in VE referring to (77). VP is similar with the mean and standard deviation from its transition kernel (79);
5. Compute the Loss by summation the following norm over x, t and ϵ
$$\|s_\theta(x_t, t) - \epsilon/\tilde{\sigma}_t\|. \tag{97}$$

Inference: Reverse denoising process

The reverse denoising process is given by

$$dx = (f(x,t) - g^2 \nabla_x \log p(x,t))dt + g(t)dB_t, \tag{98}$$

or

$$dx = (f(x,t) - \frac{1}{2} g^2 \nabla_x \log p(x,t))dt, \tag{99}$$

or

$$dx = (f(x,t) - \frac{3}{2} g^2 \nabla_x \log p(x,t))dt + \sqrt{2} g(t)dB_t, \tag{100}$$

$$\cdots \tag{101}$$

Due to obeying the same Fokker-Planck equation for $p(x,t)$

$$\frac{\partial p(x,t)}{\partial t} + \nabla \cdot (f(x,t)p(x,t)) - \frac{1}{2} g^2(t) \triangle p(x,t) = 0. \tag{102}$$

Reverse Process Proof: Setup

We prove the first reverse sampling form (98). The Fokker-Planck equation of the forward process (69) is:

$$\frac{\partial p}{\partial t} + \nabla \cdot (f(x,t)p) - \frac{1}{2}g^2 \triangle p = 0. \tag{103}$$

Let $t = T - \tau$, we have:

$$\frac{\partial p}{\partial \tau} - \nabla \cdot (f(x, T-\tau)p) + g^2 \triangle p - \frac{1}{2}g^2 \triangle p = 0. \tag{104}$$

Reverse Process Proof: Derivation

Proof (Cont).

By $\nabla \cdot \nabla = \triangle$ and $\nabla \log p = \nabla p / p$:

$$\frac{\partial p}{\partial \tau} - \nabla \cdot \left(\left(f(x, T-\tau) - g^2 \nabla \log p \right) p \right) - \frac{1}{2} g^2 \triangle p = 0. \tag{105}$$

Hence, we obtain the corresponding SDE form:

$$dx = -(f(x, T-\tau) - g^2 \nabla \log p) d\tau + g dB_\tau \tag{106}$$

By substitution $\tau = T - t$:

$$dx = (f(x, t) - g^2 \nabla \log p) dt + g dB_t \tag{107}$$

Inference recipe

The inference for VE follows the following process (VP is in a similar way)

1. Random generate a noise at time $t = 1$ as $x(1) \sim N(0, \sigma_{max}^2)$;
2. Integrate the reverse sampling equation

$$dx = \left(f(x,t) - g^2(t)s_\theta(x,t)\right)dt + g(t)dB_t \tag{108}$$

   or

$$dx = (f - \frac{1}{2}g^2 s_\theta)dt \tag{109}$$

   over the time span $[0, 1]$ to get $x(0)$.
3. Corrector process: at each internal value $x(t)$, we can relax the value $x(t)$ by a corrector, which takes the Langevin MCMC process.

The Corrector: revisit Langevin MCMC

The Langevin diffusion process is given by

$$x_i = x_{i-1} + \frac{\Delta t}{2}\nabla_x \log p(x_{i-1}) + \sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0,1). \tag{110}$$

We can prove $\pi(x)$ of the obtained data points $\{x_i\}_{i=1}^N$ converges to $p(x)$.

Proof.

The continuous form of the scheme is:

$$dx = \frac{1}{2}\nabla \log p(x)dt + dB_t. \tag{111}$$

The corresponding Fokker-Planck equation is:

$$\frac{\partial \pi(x,t)}{\partial t} + \nabla \cdot (\frac{1}{2}\nabla \log p(x)\pi(x,t)) - \frac{1}{2}\triangle \pi(x,t) = 0. \tag{112}$$

With sufficient time evolution, the distribution converges to a stable distribution where:

Proof (Cont).

Hence we have:

$$\nabla \cdot (\pi \nabla \log p - \nabla \pi) = 0, \quad \forall x. \tag{114}$$

Therefore:

$$\nabla \log p = \nabla \log \pi \tag{115}$$

given that $\pi > 0$ almost everywhere. Hence:

$$\pi^*(x) = p(x) \tag{116}$$

where π^* is the stable distribution of $\pi(x, t)$.

Outline

Additional Important Papers

The field of diffusion models has evolved rapidly with many important contributions. Here we highlight several key papers that have shaped the current understanding and practice:

- **DDPM** - The foundational work on denoising diffusion probabilistic models
- **EDM** - Elucidating design choices for better performance
- **DPM-Solver** - Fast ODE solvers for efficient sampling
- **Consistency Models** - One-step generation with diffusion quality

These papers represent significant advances in making diffusion models more practical and efficient.

Outline

DDPM: Goal & Setup

Goal. Learn a generative model $p_\theta(x_0)$ by reversing a noisy diffusion.

Latents. Introduce $x_{1:T}$ with same dimensionality as data x_0.

Reverse (sampling) chain:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \quad p(x_T) = \mathcal{N}(0, I),$$

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big).$$

Forward (fixed) diffusion:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \quad q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\big).$$

With $\alpha_t := 1 - \beta_t$, $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$:

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\, (1 - \bar{\alpha}_t) I\big).$$

DDPM: Training Objective (Variational Bound)

Optimize the NLL via a reduced-variance ELBO:

$$\mathbb{E}[-\log p_\theta(x_0)] \;\le\; \underbrace{\mathbb{E}\big[\mathrm{KL}(q(x_T|x_0)\,\|\,p(x_T))\big]}_{L_T} + \sum_{t=2}^{T} \underbrace{\mathbb{E}\big[\mathrm{KL}(q(x_{t-1}|x_t,x_0)\,\|\,p_\theta(x_{t-1}|x_t))\big]}_{L_{t-1}} - \underbrace{\mathbb{E}[\log p_\theta}_{L}$$

Here $q(x_{t-1} \mid x_t, x_0)$ is Gaussian in closed form, so each KL has a tractable expression.

Data scaling. Images scaled to $[-1, 1]$; $p_\theta(x_0 \mid x_1)$ is a discretized Gaussian decoder for likelihood evaluation.

DDPM: ϵ-Prediction Parameterization

Write the forward noising as

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Parameterize the reverse mean via a network $\varepsilon_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) \;=\; \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t) \right).$$

Sampling step (with fixed σ_t such as $\sigma_t^2 = \beta_t$ or $\tilde{\beta}_t$):

$$x_{t-1} \;=\; \mu_\theta(x_t, t) \;+\; \sigma_t\, z, \quad z \sim \mathcal{N}(0, I).$$

This connects DDPM to denoising score matching and (annealed) Langevin-like sampling.

DDPM: Simplified Loss & Algorithms

Simplified objective ("L_{simple}"):

$$\mathcal{L}_{\text{simple}}(\theta) \;=\; \mathbb{E}_{t\sim\mathcal{U}[1,T],\,x_0,\,\varepsilon}\Big\|\,\varepsilon - \varepsilon_\theta\big(\sqrt{\bar\alpha_t}\,x_0 + \sqrt{1-\bar\alpha_t}\,\varepsilon,\; t\big)\Big\|_2^2$$

Training (sketch).

1. Sample $x_0 \sim q(x_0)$, $t \sim \mathrm{Unif}\{1,\dots,T\}$, $\varepsilon \sim \mathcal{N}(0, I)$.
2. Form x_t via the forward formula; take a GD step on $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$.

Sampling (sketch).

1. $x_T \sim \mathcal{N}(0, I)$.
2. For $t = T, \dots, 1$: compute $\mu_\theta(x_t, t)$, add noise $\sigma_t z$ if $t > 1$ to get x_{t-1}.

Empirically, L_{simple} gives best sample quality; ELBO training gives better likelihoods.

DDPM: Schedules, Architecture, & Practice

Noise schedule. Typical: $T=1000$, linear β_t from 10^{-4} to 2×10^{-2} (on $[-1,1]$ data); small β_t keeps forward and reverse both Gaussian-friendly.

Network. U-Net backbone (PixelCNN++-style), group norm, sinusoidal time embedding, self-attention at medium resolution (e.g., 16×16).

Tips.

- Fixed Σ_θ (e.g., $\sigma_t^2=\beta_t$ or $\tilde{\beta}_t$) stabilizes training.
- Dropout (small) on CIFAR-10 helps; EMA on params (e.g., 0.9999).
- Evaluate with FID/IS; likelihood via discretized decoder.

DDPM: Intuitions & Connections

Progressive generation. Coarse structure emerges early (large t), details refine as $t\downarrow$.

Score-based view. ε_θ learns noise (scaled score), making the sampler resemble annealed Langevin dynamics.

Autoregressive link. The ELBO can be seen as progressive decoding with a generalized "bit ordering."

Why DDPM?

- High sample quality; stable training; flexible architectures.
- Trade-off: exact likelihoods lag some likelihood-based models, but samples excel.

Outline

Big Picture

- Goal: separate **design choices** (sampling, schedules, preconditioning, training) to simplify and speed up diffusion models.
- Key outcomes:
  - **Faster sampling** (tens of NFEs) with strong FID.
  - Modular choices: ODE view, schedule $\sigma(t)$, scaling $s(t)$, time steps $\{t_i\}$, preconditioning, loss weighting.
- Works as a **drop-in** improvement over prior models (VP/VE/DDIM/ADM).

Common ODE Framework

Probability flow ODE with schedule $\sigma(t)$ and (optional) scaling $s(t)$:

$$\frac{d\mathbf{x}}{dt} = \Big(\frac{\dot\sigma}{\sigma} + \frac{\dot s}{s}\Big)\mathbf{x} \;-\; \frac{\dot\sigma\, s}{\sigma}\,\nabla_{\mathbf{x}} \log p\big(\tfrac{\mathbf{x}}{s};\sigma\big)$$

Using denoising score matching,

$$\nabla_{\mathbf{x}} \log p(\mathbf{x};\sigma) = \frac{D(\mathbf{x};\sigma) - \mathbf{x}}{\sigma^2},$$

so the ODE is solved numerically by evaluating a **denoiser** $D_\theta(\mathbf{x};\sigma)$.

EDM choice: set $\sigma(t) = t,\ s(t) = 1 \Rightarrow$ straight(er) trajectories, stable integration.

Denoising Score Matching (Training Target)

Given clean $\mathbf{y} \sim p_{\text{data}}$, noise $\mathbf{n} \sim \mathcal{N}(0, \sigma^2 I)$,

$$D(\mathbf{y}+\mathbf{n};\sigma) \;=\; \arg\min_{D} \mathbb{E} \,\|\, D(\mathbf{y}+\mathbf{n};\sigma) - \mathbf{y} \,\|_2^2,$$

and

$$\nabla_{\mathbf{x}} \log p(\mathbf{x};\sigma) = \frac{D(\mathbf{x};\sigma) - \mathbf{x}}{\sigma^2}.$$

Train D_θ over a range of σ to match the optimal denoiser; use it at test time to integrate the ODE.

Fast Deterministic Sampling

Integrator: Heun's 2nd order (improved Euler) for better truncation error vs. Euler.

$$\text{Predict: } \mathbf{x}_{i+1}^{(E)} = \mathbf{x}_i + \Delta t_i\, f(t_i, \mathbf{x}_i) \quad \text{Correct: } \mathbf{x}_{i+1} = \mathbf{x}_i + \Delta t_i \frac{f(t_i, \mathbf{x}_i) + f(t_{i+1}, \mathbf{x}_{i+1}^{(E)})}{2}$$

Noise levels / time steps:

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \tfrac{i}{N-1}\left(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}\right)\right)^{\rho}, \quad t_i = \sigma_i, \quad \rho \approx 7.$$

Effect: reach strong FID with far fewer NFEs (e.g., $\sim$35 for CIFAR-10).

Stochastic Sampling ("Churn")

At step i (noise t_i), temporarily **increase** noise:

$$\hat{t}_i = t_i(1+\gamma_i), \quad \hat{\mathbf{x}}_i = \mathbf{x}_i + \sqrt{\hat{t}_i^2 - t_i^2}\,\varepsilon,\ \varepsilon \sim \mathcal{N}(0, S_{\text{noise}}^2 I),$$

then **one Heun step** from $\hat{t}_i$ to t_{i+1} using D_θ evaluated at the *post-churn* state $(\hat{\mathbf{x}}_i, \hat{t}_i)$.

Heuristics:

- Apply churn only for $t_i \in [t^\star_{\text{min}}, t^\star_{\text{max}}]$; set $\gamma_i = \text{Schurn}/N$, clamp to avoid excessive re-noising.
- Slightly inflate $S_{\text{noise}} > 1$ to counter over-denoising bias.

Use: helpful on more diverse data; may be unnecessary (even harmful) once training is improved.

Network Preconditioning

Represent denoiser with **preconditioned** skip connection:

$$D_\theta(\mathbf{x};\sigma) = c_{\text{skip}}(\sigma)\,\mathbf{x} \;+\; c_{\text{out}}(\sigma)\,F_\theta\big(c_{\text{in}}(\sigma)\,\mathbf{x};\; c_{\text{noise}}(\sigma)\big),$$

Design $c_{\text{in}}, c_{\text{out}}, c_{\text{skip}}$ so that:

- inputs/targets have **nearly unit variance** across σ,
- network errors are not **amplified** at large σ (robust gradients),
- F_θ smoothly interpolates between predicting signal vs. noise.

This stabilizes training and unlocks better loss scheduling.

Loss Weighting & Noise-Level Sampling

With the preconditioned form, the effective per-sample weight scales with $c_{\text{out}}(\sigma)^2$.

$$\textbf{Set} \quad \lambda(\sigma) = \frac{1}{c_{\text{out}}(\sigma)^2} \quad \Rightarrow \quad \text{balanced contributions over } \sigma.$$

Sample noise levels σ from a **log-normal** (or similar) that prioritizes the *perceptually relevant* mid-σ range:

$$\ln \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2).$$

Result: large FID gains across datasets when combined with preconditioning.

Augmentation Regularization (Non-Leaky)

- Apply standard geometric augmentations to training images *before* adding noise.
- Feed augmentation parameters as conditioning to F_θ (*non-leaky*); set to zero at inference.
- Helps small/medium datasets reduce overfitting; further FID improvements.

Headline Results & Takeaways

- **Sampling:** Heun(2) + $\sigma(t)=t$ + ρ-scheduled steps $\Rightarrow$ big NFE cuts at equal FID.
- **Training:** preconditioning + balanced loss + noise-level prior $\Rightarrow$ strong, robust models.
- **Stochasticity:** beneficial on harder data; unnecessary with strong training on simpler data.
- **Outcomes:** new SOTA-level FIDs on CIFAR-10 / ImageNet-64 with **much faster** sampling.

Outline

DPM-Solver: Motivation

- Diffusion models (DPMs) produce strong samples but are slow: **hundreds–thousands** of NFE.
- Sampling can be recast as solving a **probability flow ODE**: no stochasticity $\Rightarrow$ larger steps possible.
- **DPM-Solver** exploits the *semi-linear* structure of the diffusion ODE to deliver high-quality samples in **10–20 NFE**.
- Key idea: compute the *linear part* analytically; only approximate a special, exponentially weighted integral of the network.

DPM-Solver: From SDE to ODE

Forward SDE (VP-type example):

$$d\mathbf{x}_t = f(t)\mathbf{x}_t\,dt + g(t)\,d\mathbf{w}_t, \quad f(t) = \tfrac{d}{dt}\log\alpha_t, \quad g^2(t) = \tfrac{d}{dt}\sigma_t^2 - 2\,\tfrac{d}{dt}\log\alpha_t \cdot \sigma_t^2.$$

Probability flow ODE with learned noise predictor $\varepsilon_\theta(\mathbf{x}, t)$:

$$\frac{d\mathbf{x}_t}{dt} = f(t)\mathbf{x}_t + \frac{g^2(t)}{2\sigma_t}\,\varepsilon_\theta(\mathbf{x}_t, t), \quad \mathbf{x}_T \sim \mathcal{N}(0, \tilde{\sigma}^2\mathbf{I}).$$

Semi-linear form: linear in $\mathbf{x}_t$ plus a nonlinear NN term.

DPM-Solver: Exact Solution via Variation of Constants

Given state $\mathbf{x}_s$ at $s > t$, the exact solution of the diffusion ODE is

$$\mathbf{x}_t = e^{\int_t^s f(\tau)\,d\tau}\,\mathbf{x}_s + \int_t^s e^{\int_t^\tau f(r)\,dr}\,\frac{g^2(\tau)}{2\sigma_\tau}\,\varepsilon_\theta(\mathbf{x}_\tau, \tau)\,d\tau.$$

Introduce half-logSNR $\lambda_t := \log(\alpha_t/\sigma_t)$ (strictly decreasing in t). Using change of variables,

$$\boxed{\mathbf{x}_t = \frac{\alpha_t}{\alpha_s}\,\mathbf{x}_s \;-\; \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda}\,\hat{\varepsilon}_\theta(\hat{\mathbf{x}}_\lambda, \lambda)\,d\lambda}$$

Only the **exponentially weighted integral** of the network remains to be approximated.

DPM-Solver: Why This Helps

- **Analytic linear term** removes its discretization error (a major source of instability for large steps).
- The remaining integral matches the structure tackled by **exponential integrators**.
- Work entirely in λ (half-logSNR) space: step selection $\{\lambda_i\}$ becomes **noise-schedule invariant**.

DPM-Solver-1 (First Order)

Let time grid $t_0 = T > t_1 > \cdots > t_M = 0$, with $\lambda_i = \lambda(t_i)$, $h_i = \lambda_i - \lambda_{i-1}$.

$$\boxed{\tilde{\mathbf{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{\mathbf{x}}_{t_{i-1}} - \sigma_{t_i}\left(e^{h_i} - 1\right) \varepsilon_\theta(\tilde{\mathbf{x}}_{t_{i-1}}, t_{i-1})}$$

Fact: This update is algebraically identical to **DDIM** in λ-parameterization.

DPM-Solver-2 (Second Order)

Midpoint in λ: $s_i = t_\lambda\left(\frac{\lambda_{i-1}+\lambda_i}{2}\right)$.

$$\mathbf{u}_i = \frac{\alpha_{s_i}}{\alpha_{t_{i-1}}}\tilde{\mathbf{x}}_{t_{i-1}} - \sigma_{s_i}\left(e^{h_i/2}-1\right)\varepsilon_\theta(\tilde{\mathbf{x}}_{t_{i-1}}, t_{i-1}),$$

$$\tilde{\mathbf{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\tilde{\mathbf{x}}_{t_{i-1}} - \sigma_{t_i}\left(e^{h_i}-1\right)\varepsilon_\theta(\mathbf{u}_i, s_i).$$

Second order accuracy in λ with 2 network evaluations per step.

DPM-Solver-3 (Third Order)

Two intermediate points $s_{2i-1} = t_\lambda(\lambda_{i-1} + \frac{1}{3}h_i)$, $s_{2i} = t_\lambda(\lambda_{i-1} + \frac{2}{3}h_i)$.

Evaluate ε_θ at t_{i-1}, s_{2i-1}, s_{2i} and form finite-difference corrections D_{2i-1}, D_{2i}.

$$\tilde{\mathbf{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\tilde{\mathbf{x}}_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\varepsilon_\theta(\tilde{\mathbf{x}}_{t_{i-1}}, t_{i-1}) - \sigma_{t_i}\frac{2}{3}\left(\frac{e^{h_i} - 1}{h_i} - 1\right)D_{2i}$$

Third order accuracy in λ with 3 evaluations per step; converges in very few steps.

DPM-Solver: Choosing Step Sizes

Uniform in λ: split $[\lambda_T, \lambda_0]$ into M equal parts:

$$\lambda_i = \lambda_T + \frac{i}{M}(\lambda_0 - \lambda_T).$$

Adaptive (for larger NFE budgets): combine orders (1,2,3) with local error control in λ.

Few-step recipe (e.g., total NFE ≤ 20*):* use as many **DPM-Solver-3** steps as possible; if leftover evaluations remain, finish with a single DPM-Solver-2 (or -1) step.

DPM-Solver: Discrete-Time DPMs

Models trained at integer steps $n = 0, \ldots, N-1$ with $\tilde{\varepsilon}_\theta(\mathbf{x}_n, n)$ can be used by defining a continuous-time wrapper:

$$\varepsilon_\theta(\mathbf{x}, t) := \tilde{\varepsilon}_\theta\left(\mathbf{x}, \frac{(N-1)t}{T}\right),$$

leveraging smooth time embeddings. Then apply DPM-Solver in continuous t/λ directly.

Quality & Efficiency (Takeaways)

- **10–20 NFE** typically suffice for high-fidelity samples across standard benchmarks.
- Outperforms generic RK solvers at the same order due to **analytic linear part** and λ-**space** integration.
- Works with classifier guidance and both continuous/discrete DPMs *without retraining*.

DPM-Solver: Implementation Tips

- Precompute $\alpha_t, \sigma_t, \lambda_t$ and (if available) closed-form $t_\lambda(\cdot)$ for your noise schedule.
- Use float32 for networks but consider float64 for λ-grid arithmetic if extremely few steps are used.
- Start with uniform-λ grids; move to adaptive control only if pushing NFE below ≈ 10.
- For guidance, share predictor calls between unconditional and conditional branches when possible.

Outline

Consistency Models: Motivation

- Diffusion models (DMs) achieve SOTA quality but require many denoising steps $\Rightarrow$ slow sampling.
- **Goal:** Keep DM perks (quality, zero-shot editing, compute/quality trade-off) while enabling **fast 1-step** generation.
- **Idea:** Learn a map that sends any point on a Probability-Flow (PF) ODE trajectory back to its origin (data).

PF ODE (from SDE):

$$\mathrm{d}x_t = \left(\mu(x_t, t) - \tfrac{1}{2}\sigma(t)^2 \nabla \log p_t(x_t)\right) \mathrm{d}t \quad \Rightarrow \quad \frac{\mathrm{d}x_t}{\mathrm{d}t} = -\, t\, s_\phi(x_t, t)$$

(where $s_\phi \approx \nabla \log p_t$ under EDM-style settings)

Consistency Function & Model (Definition)

Consistency function

Given a PF ODE trajectory $\{x_t\}_{t\in[\varepsilon,T]}$, define

$$f : (x_t, t) \mapsto x_\varepsilon, \quad \text{with self-consistency } f(x_t, t) = f(x_{t'}, t') \text{ if on same trajectory.}$$

Consistency model

A neural estimator $f_\theta(x, t)$ of f trained to enforce self-consistency.

- **Boundary condition:** $f_\theta(x, \varepsilon) = x$ (identity at $t = \varepsilon$).
- Practical parameterization with skip:

$$f_\theta(x, t) = c_{\text{skip}}(t)\, x + c_{\text{out}}(t)\, F_\theta(x, t), \;\; c_{\text{skip}}(\varepsilon) = 1, \;\; c_{\text{out}}(\varepsilon) = 0.$$

Sampling: One-step and Few-step

One-step generation

$$x_T \sim \mathcal{N}(0, T^2 I), \quad x_\varepsilon = f_\theta(x_T, T).$$

Few-step (trade quality for compute): alternate denoise & noise injection

$x \leftarrow f_\theta(x_T, T)$, then for $\tau_1 > \cdots > \tau_{N-1}$: $\hat{x}_{\tau_n} \leftarrow x + \sqrt{\tau_n^2 - \varepsilon^2}\, z$, $z \sim \mathcal{N}(0, I)$; $x \leftarrow f_\theta(\hat{x}_{\tau_n}, \tau_n)$.

Time points $\{\tau_n\}$ can be greedily tuned (e.g., by FID unimodality assumption).

Training I: Consistency Distillation (CD)

Setting: Pretrained diffusion score model $s_\phi(x, t)$; discretize $[\varepsilon, T]$ into $\{t_1 = \varepsilon < \cdots < t_N = T\}$.

One-step ODE update (e.g., Heun/Euler) from $x_{t_{n+1}}$ to $\hat{x}^\phi_{t_n}$:

$$\hat{x}^\phi_{t_n} \triangleq x_{t_{n+1}} + (t_n - t_{n+1})\,\Phi(x_{t_{n+1}}, t_{n+1}; \phi).$$

CD loss: encourage equal outputs for adjacent points on same trajectory

$$\mathcal{L}^{(N)}_{\mathrm{CD}}(\theta, \theta^-; \phi) = \mathbb{E}\big[\lambda(t_n)\, d\Big(f_\theta(x_{t_{n+1}}, t_{n+1}),\ f_{\theta^-}(\hat{x}^\phi_{t_n}, t_n)\Big)\big]$$

with target EMA $\theta^- \leftarrow \mu\,\theta^- + (1-\mu)\,\theta$.

Notes: LPIPS metric and Heun solver with moderate N often work best; higher-order solvers tighten error bounds.

Training II: Consistency Training (CT, No Teacher)

Goal: Train f_θ *without* any pretrained diffusion.

$$\mathcal{L}_{\mathrm{CT}}^{(N)}(\theta, \theta^-) = \mathbb{E}_{x\sim p_{\mathrm{data}},\, z\sim\mathcal{N}(0,I),\, n}\left[\lambda(t_n)\, d(f_\theta(x + t_{n+1}z, t_{n+1}),\ f_{\theta^-}(x + t_n z, t_n))\right].$$

- Connection: as $N\to\infty$ (small Δt) with Euler, $\mathcal{L}_{\mathrm{CT}}$ matches $\mathcal{L}_{\mathrm{CD}}$ up to $o(\Delta t)$.
- Practice: progressively increase N and the EMA decay μ during training for faster convergence then higher final quality.

Consistency Models: Why it Works (Sketches)

- **Self-consistency** gives a functional fixed-point: f_θ constant along PF-ODE trajectories.
- **Boundary** $f_\theta(\cdot, \varepsilon) = \mathrm{Id}$ prevents trivial collapse.
- **CD theory:** If $\mathcal{L}_{\mathrm{CD}} = 0$ and solver local error is $\mathcal{O}(\Delta t^{p+1})$, then

$$\sup_{n,x} \|f_\theta(x, t_n) - f(x, t_n; \phi)\| = \mathcal{O}(\Delta t^p).$$

- **CT theory:** replaces teacher scores via unbiased score identities; continuous-time variant avoids bias (needs fwd-mode AD).

Zero-shot Editing via Few-step Loop

- **Denoising** across noise levels (feed x_t for any t).
- **Inpainting / Colorization / Super-resolution / Stroke-guided editing**: modify parts / constraints between denoise & re-noise steps, akin to diffusion inversion.
- **Latent interpolations:** interpolate in x_T space and map with f_θ.

Consistency Models: Quality & Speed (Selected Numbers)

- **CD (one-step)**: CIFAR-10 FID ≈ 3.55; ImageNet 64×64 FID ≈ 6.20.
- **CD (two-step)**: CIFAR-10 ≈ 2.93; ImageNet $64 \times 64 \approx 4.70$.
- **CT (direct, no teacher)**: surpasses many one-step non-adversarial baselines on CIFAR-10; improves with two-step sampling.

Zero-shot editing demonstrated on LSUN Bedroom/Cat 256^2 (colorization, SR, inpainting, stroke-guided).

Practical Tips

- Use EDM-style parameterization for $c_{\text{skip}}(t), c_{\text{out}}(t)$ and borrow U-Net backbones from diffusion.
- For CD: LPIPS metric and Heun solver with N around 18 (dataset-dependent) often best.
- For CT: schedule N and EMA μ to grow over training; continuous-time CT is elegant but needs forward-mode AD.
- Few-step sampling times $\{\tau_n\}$ can be greedily tuned (e.g., ternary search) to optimize FID.

Takeaways

- **Consistency models** provide fast **one-step** generation while enabling **few-step** trade-offs.
- Two training routes: **CD** (distill diffusion) and **CT** (train in isolation).
- Maintain diffusion-style **zero-shot editing** abilities with simpler sampling.

Outline

Questions

If you are interested in the questions, please feel free to write an email to me (hpp@bimsa.cn) with your comments.

1. Please prove that the forms (98)-(100) are equivalent with same marginal distribution $p(x,t)$ given the same initial condition;
2. Please prove that the score function $s(x,t)$ satisfying the following formula

$$\frac{\partial s}{\partial t} = \nabla(-\nabla \cdot f - \langle f, s\rangle + \frac{1}{2}g^2\|s\|^2 + \frac{1}{2}g^2\langle \nabla, s\rangle); \tag{117}$$

3. Why ISM loss is not commonly used like DSM loss in the diffusion model nowadays? Please make your comments.

Welcome inputs

Any comments?

Welcome your inputs!